Session 07 - Ownership Rule and Saving Objects - Tests

Python Contoso Vault - Session 07 of 07

Part 1 - Core part A: models.py

Goal

Record who owns each document. Let a document turn itself into a dictionary for saving, and let the `Document` class build a document back from that dictionary, so the conversion lives in one place. Write one function that guards every document by stopping any employee who is not the owner.

Open the Python shell in the `07-ownership-rule-and-saving-objects` folder and run `from models import *`, then make each call in the test plan.

Provided

Copy your own `caesar.py`, `accounts.py`, `models.py` and `vault.py` from your Session 06 folder into this folder.

Instructions

Amend the copied `models.py`. Keep the exception classes, `Employee`, the `title` property and `check_key` as they are, and make these changes in the copied `models.py`.

1. Change the `Document` constructor to take an owner as a string after the check.
   1.1. Store the owner on the object.
2. Define a method `to_dict` in `Document`.
   2.1. Return the document as a dictionary with the keys `id`, `title`, `body`, `check` and `owner`.
3. Define a class method `from_dict` in `Document` that takes a dictionary with those five keys.
   3.1. Return a new `Document` built from the dictionary.
4. Define a static method `is_valid_key` in `Document` that takes a key as an integer.
   4.1. Return `True` if the key is from 1 to 25, and `False` otherwise.
5. Define a function `require_owner` that takes a username as a string and a document as a `Document`.
   5.1. Raise `AccessDenied` with the message `Not your document.` if the username is not the document's owner.

Constraints

- Every constraint in the Session 06 `models.py` test plan still holds. A `Document` object also has the public attribute `owner`.
- `from_dict` and `is_valid_key` can be called on the class. `from_dict` called on a subclass returns an object of the subclass.
- `to_dict` returns exactly the five keys, with the title under `title`. The history, if the Session 06 Challenge was built, is not part of the dictionary.
- In `from_dict`, an empty title raises `ValidationError` and a missing key raises `KeyError`.
- The owner comparison is exact. `"Nimali"` is not the owner of a document owned by `"nimali"`.
- `require_owner` returns nothing when the username is the owner.
- No function or method prints or reads input when called.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Every Design point in the Session 06 `models.py` test plan still applies.
- Make `from_dict` a class method that builds the object through `cls`.
- Build the document in `from_dict` through the constructor.
- Make `is_valid_key` a static method.
- Give every variable and function a type hint.

Input validation

Required for the title only, as in Session 06. Assume every other value has the type the constructor names, and that a dictionary passed to `from_dict` holds values of those types.

Test plan

`document_1` is `Document(1, "Payroll", "Vdodulhv jr rxw rq 28 Qryhpehu.", "FRQWRVR-YDXOW", "nimali")`.

`dictionary_1` is `{"id": 1, "title": "Payroll", "body": "Vdodulhv jr rxw rq 28 Qryhpehu.", "check": "FRQWRVR-YDXOW", "owner": "nimali"}`.

"Same as" means equal values for all five attributes.

Test	Call or check	Expected output	Actual output	Pass
1	`document_1.owner`	`"nimali"`		
2	`document_1.to_dict()`	`dictionary_1`		
3	`document_1.title = "Payroll 2026"`, then `document_1.to_dict()["title"]`	`"Payroll 2026"`		
4	`Document.from_dict(dictionary_1)`	A `Document` the same as `document_1`		
5	`Document.from_dict(document.to_dict())` for 200 generated documents with random values	A `Document` the same as each original		
6	`Document.from_dict(dictionary_1 | {"title": ""})`	Raises `ValidationError` with the message `Title must not be empty.`		
7	`Document.from_dict` on `dictionary_1` without its `owner` key	Raises `KeyError`		
8	A subclass of `Document` calls `from_dict` on `dictionary_1`	An object of the subclass		
9	`Document.is_valid_key(k)` for `k` in 1, 13, 25	`True` for each		
10	`Document.is_valid_key(k)` for `k` in 0, 26, −3	`False` for each		
11	`document_1.is_valid_key(3)`	`True`		
12	`require_owner("nimali", document_1)`	Returns `None` and raises nothing		
13	`require_owner("kasun", document_1)`	Raises `AccessDenied` with the message `Not your document.`		
14	`require_owner("Nimali", document_1)`	Raises `AccessDenied` with the message `Not your document.`		
15	`require_owner("kasun", document_1)` inside `try`, caught by `except VaultError as error`	Caught. `str(error)` is `"Not your document."`		
16	Session 06 `models.py` tests 1 to 17, with `"nimali"` as the owner of each `Document`	The same result as each Session 06 expected result		
17	`import models`	Reads no input and prints nothing		

Done criteria

- [ ] Every test plan row gives exactly the expected result.

Part 2 - Core part B: vault.py

Goal

Give each document an owner: the employee who created it. Let an employee see, read and delete only their own documents. Hide another employee's document from their list and search, and print `Not your document.` when they try to read or delete it. Save and load through the document-to-dictionary conversion in `models.py`. Keep everything from Session 06 working.

Run `python vault.py` in the terminal from the `07-ownership-rule-and-saving-objects` folder. Type each input line and press Enter after it. Delete `accounts.json` and `documents.json` before each test that starts in an empty folder. To try one function, open the Python shell in the same folder and run `from vault import list_documents`.

Provided

Copy your own `caesar.py`, `accounts.py`, `models.py` and `vault.py` from your Session 06 folder into this folder, and amend `models.py` first. `vault.py` already defines `load_documents`, `save_documents`, `create_document`, `search_documents` and `find_document`, and runs the account menu and the five-line document menu.

Instructions

Amend the copied `vault.py`. Keep the account menu, `accounts.json` and `find_document` as they are, and make these changes in the copied `vault.py`.

1. Import `require_owner` from `models.py` as well.
2. Change `load_documents` to build each `Document` with `from_dict`.
3. Change `save_documents` to write each document with `to_dict`.
4. Change `create_document` to take an owner as a string after the key.
   4.1. Give the new document the owner.
   4.2. Give it an ID one more than the highest ID in the list, or 1 if the list is empty.
5. Define a function `list_documents` that takes the documents as a list and a username as a string.
   5.1. Return the documents the username owns, in list order.
6. Change `search_documents` to take a username as a string after the search text.
   6.1. Return only the matching documents the username owns.
7. Define a function `open_document` that takes the documents as a list, a document ID as an integer and a username as a string.
   7.1. Find the document.
   7.2. Require the username to own it.
   7.3. Return the document.
8. Define a function `delete_document` that takes the documents as a list, a document ID as an integer and a username as a string.
   8.1. Find the document.
   8.2. Require the username to own it.
   8.3. Remove it from the list.
   8.4. Return the removed document.
9. When the file runs as a program:
   9.1. Print the document menu with six lines: `1. Create`, `2. List`, `3. Read`, `4. Search`, `5. Delete`, `6. Quit`.
   9.2. On `1`, check each key with `Document.is_valid_key`, and create the document with the logged-in username as its owner.
   9.3. On `2`, list the logged-in employee's documents. Print `No documents.` if there are none.
   9.4. On `3`, open the document before reading any key.
   9.5. On `4`, search the logged-in employee's documents.
   9.6. On `5`:
      9.6.1. Read a document ID as an integer.

    9.6.2. Delete the document.
    9.6.3. Save the documents.
    9.6.4. Print `Deleted document <id>.`
  9.7. On 6, print `Goodbye.` and stop.
  9.8. Keep every step of the Session 06 program that this list does not change.

Constraints

- The functions use these parameter names, so a call with named values works: `create_document(documents, title, body, key, owner)`, `list_documents(documents, username)`, `search_documents(documents, search_text, username)`, `open_document(documents, document_id, username)` and `delete_document(documents, document_id, username)`.
- Every constraint in the Session 06 `vault.py` test plan still holds, except the menu and the `documents.json` format below.
- `documents.json` holds a JSON list of dictionaries with the keys `id`, `title`, `body`, `check` and `owner`, in list order.
- In `open_document` and `delete_document`, a missing ID raises `NotFound`.
- List and search never return or print a document another employee owns.
- Read prints `Not your document.` and reads no key when the document belongs to another employee.
- A failed delete removes nothing and saves nothing.
- Every prompt of the Session 05 `vault.py` test plan still holds. Delete reads the ID with the prompt `"ID: "`, and `"Choice: "` prints after the six menu lines. After a retry message, the prompt that asked for the value prints again. Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- The key prompt keeps the Session 06 messages: `Please enter a whole number.` and `Key must be from 1 to 25.`
- `list_documents`, `open_document` and `delete_document` neither print nor read input.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Every Design point in the Session 06 `vault.py` test plan still applies.
- Name the keys of `documents.json` only in `to_dict` and `from_dict` in `models.py`.
- Check ownership in `open_document` and `delete_document` with `require_owner`.
- Keep exactly one `except VaultError`.
- Check each key with `Document.is_valid_key`.
- Save the documents straight after each delete, before `Deleted document <id>.` prints.
- Give every variable and function a type hint.

Input validation

Required for every key and ID, as in Session 06, and for the title through `Document`. Not required for usernames, passwords, bodies and search texts. Assume `documents.json` and `accounts.json` are missing or were written by this program.

Test plan

Function tests

`dictionary_1` is `{"id": 1, "title": "Payroll", "body": "Vdodulhv jr rxw rq 28 Qryhpehu.", "check": "FRQWRVR-YDXOW", "owner": "nimali"}`.

`dictionary_2` is `{"id": 2, "title": "Kandy rent", "body": "Vlyoj zqqtnp cpye td Cd 150,000.", "check": "NZYEZDZ-GLFWE", "owner": "nimali"}`.

`dictionary_3` is `{"id": 3, "title": "Galle rent", "body": "Lfqqj xmtu wjsy nx Wx 90,000.", "check": "HTSYTXT-AFZQY", "owner": "kasun"}`.

`document_1` to `document_3` are the `Document` objects built from `dictionary_1` to `dictionary_3`. Here, `documents` starts as `[document_1, document_2, document_3]` in each test, and `path` names a file in an empty folder. "Same as" means equal values for all five attributes.

Test	Call	Expected output	Actual output	Pass
1	`create_document(new_documents, "Payroll", "Salaries go out on 28 November.", 3, "nimali")`, with `new_documents` as `[]`	A `Document` the same as `document_1`. `new_documents` holds it		
2	`create_document(new_documents, "Galle rent", "Galle shop rent is Rs 90,000.", 5, "kasun")`, with `new_documents` as `[document_2]`	A `Document` the same as `document_3`		
3	`create_document(new_documents, "Kandy rent", "Kandy office rent is Rs 150,000.", 11, "nimali")`, with `new_documents` as `[document_3, document_1]`	A `Document` with ID 4		
4	`create_document(new_documents, "", "Body text", 3, "nimali")`, with `new_documents` as `[]`	Raises `ValidationError`. `new_documents` is still `[]`		
5	`save_documents([document_1, document_3], path)`	The file holds the JSON list `[dictionary_1, dictionary_3]`		
6	Then `load_documents(path)`	A list of two `Document` objects, the same as `document_1` and `document_3`		
7	`load_documents(path)`, with no file at `path`	`[]`		
8	`list_documents(documents, "nimali")`	`[document_1, document_2]`		
9	`list_documents(documents, "kasun")`	`[document_3]`		
10	`list_documents(documents, "ravi")`	`[]`		
11	`search_documents(documents, "rent", "nimali")`	`[document_2]`		
12	`search_documents(documents, "rent", "kasun")`	`[document_3]`		
13	`search_documents(documents, "3", "nimali")`	`[]`		
14	`search_documents(documents, "", "nimali")`	`[document_1, document_2]`		
15	`open_document(documents, 2, "nimali")`	`document_2`		
16	`open_document(documents, 3, "nimali")`	Raises `AccessDenied` with the message `Not your document.`		
17	`open_document(documents, 9, "nimali")`	Raises `NotFound` with the message `No such document.`		
18	`delete_document(documents, 2, "nimali")`	`document_2`. `documents` is `[document_1, document_3]`		
19	`delete_document(documents, 3, "nimali")`	Raises `AccessDenied` with the message `Not your document.` `documents` is unchanged		
20	`delete_document(documents, 9, "nimali")`	Raises `NotFound` with the message `No such document.` `documents` is unchanged		
21	Session 06 `vault.py` function tests 6 to 9	The same result as each Session 06 expected result		

Program tests

Each test runs `python vault.py` in an empty folder, once or twice in a row. "Sign up Nimali", "Log in Nimali", "Create Payroll", "Create Kandy rent" and "Create Galle rent" are as in the Session 05 `vault.py` test plan. "Sign up Kasun" stands for the three input lines `"1"`, `"kasun"`, `"kandy-rain-88"`, and "Log in Kasun" for `"2"`, `"kasun"`, `"kandy-rain-88"`. Every run prints the account menu and the prompt `"Choice: "` before each account choice, and the document menu and the prompt `"Choice: "` before each document choice. Every run ends with the `Goodbye.` line. The expected output cells leave out the menu lines, the prompts and the `Goodbye.` line. They list the remaining output lines, such as `Saved document 1.`

Test	First run input lines	Second run input lines	Expected output	Actual output	Pass
22	Sign up Nimali, Sign up Kasun, Log in Nimali, Create Payroll, Create Kandy rent, 6	Log in Kasun, Create Galle rent, 2, 3, 1, 4, rent, 5, 2, 6	`Welcome, kasun.`, `Saved document 3.`, `3. Galle rent`, `Not your document.`, `3. Galle rent`, `Not your document.` File: `[dictionary_1, dictionary_2, dictionary_3]`		
23	Sign up Nimali, Sign up Kasun, Log in Nimali, Create Payroll, 6	Log in Kasun, 2, 4, pay, 4, 1, 6	`Welcome, kasun.`, `No documents.`, `No matches.` twice File: `[dictionary_1]`		
24	Sign up Nimali, Log in Nimali, Create Payroll, Create Kandy rent, 5, 1, 2, 5, 1, 6	No second run	`Account created.`, `Welcome, nimali.`, `Saved document 1.`, `Saved document 2.`, `Deleted document 1.`, `2. Kandy rent`, `No such document.` File: `[dictionary_2]`		
25	Sign up Nimali, Log in Nimali, Create Payroll, Create Kandy rent, 5, 1, 6	Log in Nimali, 2, 6	`Welcome, nimali.`, `2. Kandy rent` File: `[dictionary_2]`		
26	Sign up Nimali, Log in Nimali, Create Payroll, Create Kandy rent, 5, 1, Create Galle rent, 2, 6	No second run	`Account created.`, `Welcome, nimali.`, `Saved document 1.`, `Saved document 2.`, `Deleted document 1.`, `Saved document 3.`, `2. Kandy rent`, `3. Galle rent` File: `[dictionary_2, dictionary_3]`, with `"nimali"` as the owner in `dictionary_3`		
27	Sign up Nimali, Log in Nimali, 5, x, 1, 6	No second run	`Account created.`, `Welcome, nimali.`, `Please enter a whole number.`, `No such document.` File: none		
28	Sign up Nimali, Sign up Kasun, Log in Nimali, Create Payroll, 6	Log in Kasun, 5, 1, 6	`Welcome, kasun.`, `Not your document.` File: `[dictionary_1]`		
29	`import vault`	No second run	Reads no input and prints nothing File: none		
30	Each Session 06 program test, 11 to 17, with each document menu choice 5 changed to 6	As in the Session 06 test, with each 5 changed to 6	As in the Session 06 test, with the six-line document menu File: as in the Session 06 test, with `"nimali"` as the owner of each document		

"File" is `documents.json` after the last run. In every program test, the program ends after `Goodbye.` and shows no Python error message.

Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ The script runs from the terminal with `python vault.py`.

Part 3 - Core part C: two_accounts.py

Goal

Prove the ownership rule with two employees, Nimali and Kasun. Check that Kasun cannot list, read, search or delete Nimali's document, and that Nimali still can. Stop with an error if any check fails. Print `All checks passed.` if every check passes.

Run `python two_accounts.py` in the terminal from the `07-ownership-rule-and-saving-objects` folder.

Provided

Build `two_accounts.py` beside your own amended `vault.py` and `models.py` in this folder.

Instructions

1. Import the vault functions it needs from `vault.py`, and `Document` and `AccessDenied` from `models.py`.
2. When the file runs as a program:
   2.1. In a list in memory, create a document for `nimali` with the title `"Payroll"`, the body `"Salaries go out on 28 November."` and the key 3. Create one for `kasun` with the title `"Galle rent"`, the body `"Galle shop rent is Rs 90,000."` and the key 5.
   2.2. Check that Kasun's list leaves out Nimali's document.
   2.3. Check that Kasun's search for `"Payroll"` leaves out Nimali's document.
   2.4. Check that opening Nimali's document as Kasun raises `AccessDenied`.
   2.5. Check that deleting Nimali's document as Kasun raises `AccessDenied` and leaves it in the list.
   2.6. Check that Nimali can list, search, open and delete her own document.
   2.7. Print `All checks passed.`

Constraints

- The script calls only `create_document`, `list_documents`, `search_documents`, `open_document` and `delete_document` from `vault.py`, with the parameter names that Core part B fixes. It imports only `Document` and `AccessDenied` from `models.py`.
- A failed check stops the script with `AssertionError`.
- A check that expects `AccessDenied` fails when no error is raised.
- The checks run only when the file runs as a program. Importing the file runs no check.
- The script reads no input and writes no file.
- The only output of a passing run is `All checks passed.`

Design

The following rules say how to write the code. Follow them.

- Write each check as an `assert`.
- Give every variable a type hint.

Input validation

None required. The script reads no input.

Test plan

Each test runs in this folder beside your own `vault.py` and `models.py`.

Test	Run or call	Expected output	Actual output	Pass
1	`python two_accounts.py` with the folder's own `vault.py` and `models.py`	Prints `All checks passed.` and exits with code 0		
2	`import two_accounts`	Prints nothing and raises nothing		
3	Any test above	No file appears in the folder		

Done criteria

☐ Every test plan row gives exactly the expected result.

Part 4 - Challenge: loyalty.py

Goal

Build a loyalty card program for a grocery shop. Give each customer a card. Let a customer earn points by shopping, spend points for a discount, and give points to a family member. Read one command from the shop owner on each line, and answer it with one line. Save the cards, so they are still there when the program starts again. Plan how the program works inside on your own. Match the commands and messages in the test plan exactly.

Run `python loyalty.py` in the terminal from the `07-ownership-rule-and-saving-objects` folder. Type each input line and press Enter after it. Delete `cards.json` before each test that starts in an empty folder. To try one function, open the Python shell in the same folder and run `from loyalty import load_cards`.

Instructions

1. A class `Card` whose objects have the attributes `number` as a string, `name` as a string and `points` as an integer.
2. A function `load_cards` that takes the path of a JSON file as a string and returns a dictionary that maps each card number to its `Card`. It returns an empty dictionary if the file does not exist.
3. A function `save_cards` that takes that dictionary and the path of a JSON file as a string, and writes every card to the file as JSON.
4. When the file runs as a program, it loads the cards from `cards.json`, then reads one command per line until `quit`.
   4.1. A command is words separated by spaces. The first word names the command, and the other words are its values.
   4.2. Answer every command with exactly one line.
   4.3. Print `Invalid command.` for an unknown first word, an empty line, or a command with the wrong number of values. Check this before anything else.
   4.4. Check each command's refusals below in the order listed, and print only the first that applies.
5. Command type #1: `join <number> <name>` opens a card with 0 points. A card number is 10 digits that start with `07`, such as `0771234567`. The name is every word after the number, so `join 0771234567 Amaya Perera` gives the name `Amaya Perera`. The name is kept as typed.
   5.1. On success, print `Card <number> opened for <name>.`
   5.2. Print `Card numbers are 10 digits starting with 07.` if the number has the wrong form.
   5.3. Print `Card <number> already exists.` if the number already has a card.
6. Command type #2: `buy <number> <rupees>` records a purchase and adds 1 point for each full Rs 100. `buy 0771234567 2450` adds 24 points.
   6.1. On success, print `<name> earned <points> points. Balance: <balance>.`
   6.2. Print `Amounts must be whole numbers from 1.` if the rupees are not a whole number from 1 upward, such as `0`, `-5` or `abc`.
   6.3. Print `No such card: <number>.` if the number has no card.
7. Command type #3: `redeem <number> <points>` takes points off the card.
   7.1. On success, print `<name> redeemed <points> points. Balance: <balance>.`
   7.2. Print `Amounts must be whole numbers from 1.` if the points are not a whole number from 1 upward.
   7.3. Print `No such card: <number>.` if the number has no card.
   7.4. Print `Not enough points. Balance: <balance>.` if the card holds fewer points.
8. Command type #4: `transfer <from> <to> <points>` moves points from one card to another.
   8.1. On success, print `Moved <points> points from <from name> to <to name>.`
   8.2. Print `Amounts must be whole numbers from 1.` if the points are not a whole number from 1 upward.
   8.3. Print `No such card: <from>.` if the from number has no card, then `No such card: <to>.` for the to number.
   8.4. Print `Cannot transfer to the same card.` if both numbers are the same.
   8.5. Print `Not enough points. Balance: <balance>.` with the from card's balance if it holds fewer points.
9. Command type #5: `balance <number>` shows the card's points.
   9.1. Print `<name> has <balance> points.`
   9.2. Print `No such card: <number>.` if the number has no card.

10. Command type #6: `quit` stops the program.
    10.1. Print `Goodbye.`

Constraints

- No card balance is ever below zero.
- Points are created only by `buy` and removed only by `redeem`. A `transfer` never creates or loses a point.
- Every message says `point` for exactly 1 point and `points` for any other number, as in `earned 1 point` and `has 0 points`. The number after `Balance:` ends the message, followed only by a full stop.
- A card number is kept as text, so its first `0` stays.
- Cards are saved in `cards.json`, so every change is still there when the program starts again. How the JSON lays out each card is your choice.
- Only a command that changes a card writes the file. A refused command changes no card.
- Read each command with a prompt on the same line, as in `input("Command: ")`. The prompt is `"Command: "`, its label, a colon and one trailing space. It prints before every command, including after `Invalid command.` Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- The program prints nothing beyond the prompt and the messages above.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Save the cards straight after every command that changes a card, so a power cut loses nothing.
- Give every variable and function a type hint.

Input validation

Required for every command, as the instructions and constraints set out. Assume the input always ends with `quit`, and that `cards.json` is missing or was written by this program.

Test plan

Each test runs `python loyalty.py` in an empty folder, once or twice in a row. "Join Amaya" stands for `join 0771234567 Amaya Perera`, and "Join Ruwan" for `join 0779876543 Ruwan Perera`. Every run prints the prompt `"Command: "` before each command. The cells leave out the prompts and the `Card <number> opened for <name>.` line that each Join prints. Every run ends with the input line `quit` and the output line `Goodbye.`, which the cells leave out.

Test	First run input lines	Second run input lines	Expected output	Actual output	Pass
1	Join Amaya, `buy 0771234567 2450`, `balance 0771234567`	No second run	`Amaya Perera earned 24 points. Balance: 24.`, `Amaya Perera has 24 points.`		
2	Join Amaya, `buy 0771234567 2450`, `redeem 0771234567 30`, `redeem 0771234567 20`	No second run	`Amaya Perera earned 24 points. Balance: 24.`, `Not enough points. Balance: 24.`, `Amaya Perera redeemed 20 points. Balance: 4.`		
3	Join Amaya, Join Ruwan, `buy 0771234567 1000`, `transfer 0771234567 0779876543 4`, `balance 0771234567`, `balance 0779876543`	No second run	`Amaya Perera earned 10 points. Balance: 10.`, `Moved 4 points from Amaya Perera to Ruwan Perera.`, `Amaya Perera has 6 points.`, `Ruwan Perera has 4 points.`		
4	Join Amaya, `buy 0771234567 2450`	`balance 0771234567`	`Amaya Perera has 24 points.`		
5	Join Amaya, `buy 0771234567 99`, `buy 0771234567 100`, `balance 0771234567`	No second run	`Amaya Perera earned 0 points. Balance: 0.`, `Amaya Perera earned 1 point. Balance: 1.`, `Amaya Perera has 1 point.`		
6	Join Amaya, `join 0771234567 Nimal`, `join 12345 Nimal`, `join 0871234567 Nimal`	No second run	`Card 0771234567 already exists.`, then `Card numbers are 10 digits starting with 07.` twice		
7	Join Amaya, `buy 0771234567 0`, `buy 0771234567 -5`, `buy 0771234567 abc`	No second run	`Amounts must be whole numbers from 1.` three times		
8	`sell 0771234567 5`, empty line, `buy 0771234567`, `balance`	No second run	`Invalid command.` four times		
9	`buy 0771234567 500`, `balance 0771234567`	No second run	`No such card: 0771234567.` twice		
10	Join Amaya, `buy 0771234567 1000`, `transfer 0771234567 0779876543 4`, `balance 0771234567`	No second run	`Amaya Perera earned 10 points. Balance: 10.`, `No such card: 0779876543.`, `Amaya Perera has 10 points.`		
11	Join Amaya, Join Ruwan, `buy 0771234567 1000`, `transfer 0771234567 0779876543 11`, `transfer 0771234567 0771234567 1`	No second run	`Amaya Perera earned 10 points. Balance: 10.`, `Not enough points. Balance: 10.`, `Cannot transfer to the same card.`		
12	Join Amaya, Join Ruwan, `buy 0771234567 1000`, `transfer 0771234567 0779876543 1`, `balance 0779876543`	No second run	`Amaya Perera earned 10 points. Balance: 10.`, `Moved 1 point from Amaya Perera to Ruwan Perera.`, `Ruwan Perera has 1 point.`		
13	Join Amaya, Join Ruwan, `buy 0771234567 1000`	`transfer 0779876543 0771234567 1`, `redeem 0771234567 10`	`Not enough points. Balance: 0.`, `Amaya Perera redeemed 10 points. Balance: 0.`		

In every program test, the program ends after `Goodbye.` and shows no Python error message.

Done criteria

☐ Every test plan row gives exactly the expected result.

Marking

This take-home is marked out of 100, with a pass mark of 60.

Part	Tests	Design	Total
Core	64	16	80
Challenge	16	4	20
Total	**80**	**20**	**100**

Guidelines

- Test marks show that the program works. The instructor's tests run it on every row of the test plan, plus further cases.
- Design marks show that the code is built well. The instructor reads the code against each Design rule.
- A pass needs 60 marks from the Core alone. Challenge marks count towards the total once the Core reaches 60.
- Marking uses the last commit pushed by 11:59 PM six days after this session.
- The submission is one GitHub link to the fork, posted on the portal.